

FIRE 7: Automatic Reduction with Modular Approach

Alexander V. Smirnov^{a,b,*}, Mao Zeng^c

^a*Research Computing Center, Moscow State University,
119992 Moscow, Russia*

^b*Moscow Center for Fundamental and Applied Mathematics, 119992, Moscow, Russia*

^c*Higgs Centre for Theoretical Physics, University of Edinburgh,
James Clerk Maxwell Building, Peter Guthrie Tait Road, Edinburgh, EH9 3FD,
United Kingdom*

Abstract

FIRE7 is a major update to the FIRE program for integration-by-parts (IBP) reduction of Feynman integrals. A large part of improvements is related to the automatic reduction and reconstruction with the modular arithmetic approach, while the performance of the classical rational polynomial approach is also significantly increased. An improved presolve algorithm performs Gaussian elimination to simplify IBP identities before substituting numerical indices as in the Laporta algorithm. Various new command line tools are included to facilitate tasks such as applying an IBP reduction table to reduce a loop integrand as a linear combination of individual integrals.

Keywords: Feynman integrals, Integrate by parts, Computer algebra

*Corresponding author

Email addresses: `asmirnov80@gmail.com` (Alexander V. Smirnov),
`mao.zeng@ed.ac.uk` (Mao Zeng)

PROGRAM SUMMARY

Program title: FIRE, version 7 (FIRE 7)

CPC Library link to program files: (to be added by Technical Editor)

Developer's repository link: <https://gitlab.srcc.msu.ru/feynmanintegrals/fire.git>

Code Ocean capsule: (to be added by Technical Editor)

Licensing provisions: GPLv2

Programming language: Wolfram Mathematica 8.0 or higher, C++17

Supplementary material: see linked repository for installation instructions and examples

Journal reference of previous version: <https://doi.org/10.1016/j.cpc.2024.109261>

Does the new version supersede the previous version? Yes.

Reasons for the new version: The new version brings major improvements to the performance of IBP reduction with modular arithmetic, enhanced parallelization, and a variety of convenient user tools.

Summary of revisions: The new version enables MPI parallelization of finite-field IBP reduction runs and reconstruction of analytic results. Many performance improvements, especially those relevant for the modular arithmetic approach, are delivered. Linear combinations of Feynman integrals, e.g. as found in loop integrands, can be directly reduced. Various new utilities are provided for manipulating reduction tables.

Nature of problem: The use of modular arithmetic is a major advance in efficient and parallelized reduction of Feynman integrals, and therefore a high-quality implementation as part of the FIRE program is desired. Many “peripheral” operations outside of the IBP reduction computation itself, such as the application of IBP tables to reduce a loop integrand, need efficient and convenient implementations.

Solution method: Major new parallelized algorithms for finite-field IBP reduction are implemented. Gaussian elimination prior to seeding, i.e. a “presolve” step, enhances the efficiency of IBP reduction. Multiple numerical parameter points can be run simultaneously to significantly reduce resource overhead. *References:*

- [1] <https://home.bway.net/lewis/>, free-ware with some restrictions;
- [2] <https://doi.org/10.26089/NumMet.v24r425>, open source;
- [3] <https://flintlib.org/>, open source;
- [4] <https://symbolica.io/>, commercial software with free licenses for students and hobbyists

1. Introduction

The evaluation of Feynman integrals is a fundamental component of perturbative quantum field theory computations. A key technique in this process is integration-by-parts (IBP) reduction [1, 2], which allows all integrals within a given family (characterized by a set of propagators) to be expressed in terms of a finite set of simpler, so-called master integrals. This method leverages the fact that total derivatives vanish under dimensional regularization, giving rise to linear relations between integrals, known as IBP identities, which are often supplemented by Lorentz invariance relations [3]. Typically, the master integrals are chosen for their relative simplicity, involving low-degree numerators and propagators raised to low powers.

In all but the most basic cases, performing reduction requires combining IBP identities in nontrivial ways. A systematic procedure for this is the Laporta algorithm [4]. Several public implementations of this algorithm exist, including `AIR` [5], `Reduze` [6, 7], `LiteRed` [8, 9, 10], `FIRE` [11, 12, 13, 14], and `Kira` [15, 16, 17]. The authors of this paper have been developing `FIRE` for years, a program which has been widely used in state-of-the-art computations in high-energy physics. The Laporta algorithm works by generating all IBP identities up to a specified complexity, followed by solving the resulting system of linear equations via Gaussian elimination. These equations have polynomial coefficients in the spacetime dimension d and in kinematic parameters like Mandelstam variables and masses, which turn into rational functions during Gaussian elimination. To avoid a blow-up in expression size, it is critical to simplify these rational functions — typically by reducing them to a single fraction with co-prime numerator and denominator throughout all the reduction steps.

Alternative strategies to the Laporta algorithm include deriving symbolic reduction rules that depend on propagator powers and irreducible scalar products. Such rules can be obtained using heuristic methods (as in `LiteRed` [8, 9]), techniques involving Gröbner bases [18, 19, 20, 21, 22, 23, 24], generating functions [25, 26, 27], and variants [28, 29] of the syzygy equation method also mentioned later. These rules reduce more complicated integrals to simpler ones according to a prescribed ordering, and repeated application is usually required to fully reduce an integral to master integrals. Again, rational function simplification plays a vital role in managing intermediate expression complexity. Methods based on syzygy equations [30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40] aim to shrink the size of the linear

system, while techniques employing block-triangular forms [41, 42, 43] help to organize the system more efficiently. There is also work on bypassing the need to solve linear systems of IBP equations [44, 45]. Nevertheless, the simplification of rational functions remains a central computational challenge across all these approaches.

An alternative approach to reduction involving rational functions of many variables is performing numerical evaluations followed by analytic reconstruction. However, floating-point arithmetic is unsuitable due to the need for exact coefficients, and rational arithmetic—though exact—requires costly arbitrary-precision computations. To overcome this, Refs. [46, 47, 48] proposed using finite field arithmetic for IBP reductions. Finite fields offer significant efficiency advantages, as elements are represented by machine-size integers with exact arithmetic. While finite fields entail information loss, this can be resolved by reconstructing rational numbers from evaluations in several fields using the Extended Euclidean Algorithm [49, 50], based on the Chinese Remainder Theorem [51].

Once numerical data are obtained, the main task is reconstructing the target function (the "black box") from its sampled values. Univariate interpolation techniques such as Newton and Thiele methods [52] are classical, but multivariate reconstruction is more recent. Sparse multivariate interpolation—assuming a bounded number of terms—has been developed in Refs. [53, 54, 55, 56, 57, 58]. Dense reconstructions, without such bounds, are less common. A Thiele-based generalization was implemented in FIRE6 [59], but it struggles beyond two variables. The most effective multivariate approach is based on homogeneous scaling [57], adopted in `FiniteFlow` [60] and `FireFly` [61], the latter integrated with Kira 2.0 [17] and Kira 3.0 [62]. In [63] we suggested a balanced reconstruction approach as an alternative to homogeneous scaling, later combined with the Zippel algorithm to take advantage of sparsity [64], which is integrated into this new version of FIRE.

In this paper we present a new version of FIRE with automates the modular arithmetic approach, including reduction and subsequent reconstruction, “with one command” with MPI parallelization. In section 2 we describe our approach. In section 3 we describe other new improvements in FIRE7. In section 4 the installation and usage of FIRE is presented. In section 5 we provide examples and benchmarks.

Note: for the rest of the paper, unless otherwise specified, file locations are relative to the folder `fire/FIRE7`.

2. Modular approach

2.1. Overview of the approach

One of the most important features presented in the new version of FIRE is the ability to automatically perform reduction with the modular approach¹ with the subsequent reconstruction of coefficients, i.e. inferring analytic expressions from a large number of exact numerical evaluations in finite fields. With this feature we aim to assist the user as much as possible with reconstruction, especially in the case with multiple variables, where a manual reconstruction process would become very cumbersome requiring multiple reconstruction sub-steps (Thiele reconstruction, Newton reconstruction etc.) proceeding with variables one by one.

This approach relies on the following parts of the FIRE package:

- The FIRE reduction program itself, specifically the `bin/FIRE7p` binary (where `p` refers to the prime variant) which performs reduction in a single probe point, i.e. a set of values given to space-dimension and kinematic invariants and a large prime number chosen for reduction, or alternatively the `FIRE7mp` binary (where `mp` refers to the multiprime variant) which simultaneously performs reductions in multiple probe points;
- The `bin/reconstruction` tool which takes some tables generated by FIRE and created a reconstructed table (each call performs either reconstruction for the analytic dependence on one more variable or rational reconstruction, i.e. recovering rational numbers from finite-field images.);
- An MPI program `bin/FIRE7_MPI` that calls both the FIRE reduction program and the reconstruction tool in order to obtain the reconstructed result automatically.

The reason to use such an approach was discussed many times in papers on Feynman integrals. The modular approach is an alternative to the classical (rational polynomial) approach where the reduction is performed with coefficients being rational functions of many variables. Within the classical

¹To disambiguate, “modular” always refers to modular arithmetic in this paper.

approach almost all the time consumed by the program is spent on simplifying coefficients with an external program. (In the past `fermat` was the dominant choice, but now the `FLINT` library has proven to be more effective and is made the default in `FIRE`.) The time needed for reduction as well as RAM required can be too large to fit into a reasonable session, so there is always a risk of running out of RAM or hardware failures making one lose months of calculation if it is interrupted in an unexpected state.

The modular approach is much more resistant to possible interruptions since one might need millions of intermediate reduction runs, each of which results in a separate table file stored on disk. Moreover, the modular approach can efficiently take use of multiple machines working on the same task, especially in the case of supercomputers. However the MPI-parallelized program is not restricted to clusters and can be launched on a single machine as well.

2.2. Modular reduction in *FIRE*

To manually call modular reduction with `FIRE7p` one should create a `config` file as usual (see the full `config` syntax in Section 4.2) and then `FIRE7p` should be called with the `--variables` option passed. The variable values should be underscore-separated, and the last position is the index of the large prime number used as the finite-field modulus. The primes used by `FIRE` are stored in the `sources/tools/primes.cpp` file, where at position 0 we use the number 2017 (purely for testing purposes), followed by large primes, 127 of which are close to 2^{64} at positions 1 to 127 and are recommended for regular use as the modulus. The first six primes in this list are: 18446744073709551557, 18446744073709551533, 18446744073709551521, 18446744073709551437, 18446744073709551427, 18446744073709551359. There are also 128 primes close to $2^{63.5}$ at positions 128 to 255 and 128 primes close to 2^{31} at positions 256 to 383. Variables values can be numbers and also they accept the b^e syntax for given base b and exponent e . It is also possible to automatically use the prime numbers at positions 128 onward as base values for variables, using the notation $p0, p1, \dots$, denoting prime numbers from the `primes.cpp` file at positions 128, 129, $\dots$ (note the offset). Here are some valid examples:

- `bin/FIRE7p --variables 80_90_100_1 --config examples/doublebox3N --parallel`

This uses $t = 80, s = 90, d = 100$, with the modulus as the prime at position 1, 18446744073709551557.

- `bin/FIRE7p --large_variables --variables 80^1_90^1_100^1_1 --config examples/doublebox3N --parallel`

This is the same as the previous one, as the exponents for the variables are 1. The `--large_variables` option is required for using the exponent notation, as large intermediate numbers can be produced by exponentiation before modding by the prime modulus.

- `bin/FIRE7p --variables p1^5_p2^6_p3^7_2 --config examples/doublebox3N --parallel`

This uses $t = 13043817825332782193^5, s = 13043817825332782171^6, d = 13043817825332782093^7$, where the base values are primes at positions 128, 129, and 130, with the modulus as the prime at position 2, 18446744073709551533.

FIRE7p saves reduction results in `tables` files. Since one is going to call FIRE7p multiple times, the output table filenames are different — before the final `.tables` extension, FIRE7p appends the variable values as it was passed by the `--variables` option. For example, with the second variant above, the table filename will be `doublebox_80^1_90^1_100^1_1.tables` assuming that the `config` file has specified the output filename `doublebox.tables`.

2.3. Reconstruction tool

The reconstruction tool `bin/reconstruction` takes a number of tables, performs reconstruction of all coefficients, and saves result as a new table file. For most users, the MPI wrapper covered in Section 2.4 automates the reconstruction steps in a more convenient manner, while internally calling `bin/reconstruction`, so this subsection can be skipped for a first reading. For users who want more manual control, e.g. generating tables and performing reconstructions step-by-step, `bin/reconstruction` can be used directly, assuming familiarity with the balanced Zippel reconstruction algorithm [63, 64]. There is a number of reconstruction methods this tool supports, chosen by the `--method` command line option:

- **rational**: rational reconstruction from finite-field images to rational numbers. It is advisable to have rational reconstruction as the final reconstruction step after all other reconstructions, as explained in e.g. [63];

- **thiele**: Thiele reconstruction by one variable; in practice should be used only for coefficients not depending on other variables, i.e. for the first variable and to create balancing tables [63, 64];
- **newton**: newton reconstruction by one variable; implemented but not used in pure form by FIRE7_MPI;
- **numeratorNewton**: assuming the last reconstruction variable is the space-time dimension d and masters are chosen in a proper way [65, 66], we know that the denominators of all master coefficients can be factorized into a function of d and a function of other (kinematic) variables. Thus after a Thiele reconstruction by d for one set of values of other variables, the d -dependent part of the denominator is known, and further reconstructions can be performed with this knowledge. This is because knowing the denominator in d , one can multiply by it and use Newton reconstruction for the numerator when knowing the reconstruction value in the kinematic variables;
- **balancedNewton**: the method used to proceed from k reconstructed variables to $k + 1$ with the use of balancing tables so that numerator and denominator are reconstructed independently [63];
- **balancedZippel**: having a reconstructed function of k variables for one value of the subsequent variable, this method reconstructs the function for k variables and a different value of the subsequent variable; not used directly by FIRE7_MPI;
- **balancedZippelNewton**: most efficient method to proceed from k reconstructed variables to $k + 1$ reconstructed variables (especially for multiple variables and thus sparse coefficients), as reconstruction with **balancedZippel** followed by **balancedNewton** without storing intermediate results.

An experimental Mathematica interface to the reconstruction tool is `mm/FireReconstruct.wl`. The Mathematica notebook `examples/manualReconstructionExample.nb` explains the usage and demonstrates how one could use the reconstruction tool for general purposes (not necessary for IBP). The notebook transparently shows the shell commands called under the hood, which can also help with learning the manual usage of the reconstruction tool.

2.4. MPI wrapper

The so-called MPI wrapper, the binary `bin/FIRE7_MPI` is a program that is designed to automatically run modular reduction and subsequent reconstructions. The full launch syntax will be described later, and here we give a general overview. First of all, this program uses MPI, i.e. Message Passing Interface, a protocol that allows communication between multiple processes that can be launched on different computers at the same time. A native way to launch the program locally is

```
mpirun -np <n_processes> bin/FIRE7_MPI <arguments>
```

where `<n_processes>` should be replaced with the number of processes to run concurrently. (When running on one local computer, typically one can set it equal to the number of cores or exceeding it by one since the master process does not utilize CPU much). When running on a cluster with multiple machines for one reduction, one should refer to the local cluster documentation on how MPI programs are launched. The basic syntax to call the wrapper is, e.g.

```
mpirun -np 5 bin/FIRE7_MPI -Z --reconstruct -P 3 -I \
80_90_100 -S -c examples/doublebox3N
```

Here we provide starting variable values $t = 80, s = 90, d = 100$, use the Zippel approach with the `-Z` flag, ask the code to do the reconstruction and set a limit of a maximum of 3 prime values. Also we pass the important information that last variable can be split (factorized) in denominators [65, 66] with the `-S` flag. This call is sufficient to initiate IBP reduction computations at numerical parameter values, and the run completes when the desired reduction table is reconstructed.

The order in which the variables are placed is very important. As a start we suggest to start running FIRE with any order and to detect Thiele “limits” — the number of probe points to perform a one-dimensional Thiele reconstruction by the corresponding variable. We recommend reordering variables in the decreasing order of Thiele limits, i.e. starting from variables that need a large number of probe points in Thiele reconstruction. However, if one can choose a basis of master integrals such that the denominators by d are split, it is recommended to put d at the last place, as the smaller Newton limit will be used for it instead.

2.5. Reducing for multiple numerical probes simultaneously

Working with the modular approach has its disadvantages. One of those is that the overheads for solving the linear system of equations become

too large compared with the actual solving, i.e. performing modular arithmetic operations during Gaussian elimination. To decrease the overheads and also to decrease the number of table files stored on disk, we created a FIRE variant that performs reduction at multiple probe points at the same time. By default FIRE7 uses 16 points in this variant, and this can be changed by the `configure` script which requires a full rebuild of FIRE.

To use such a reduction program one should call the `bin/FIRE7mp` binary (where `m` stands for multiple) with almost the same syntax, but adding a plus symbol at the places where the variables differ. The following examples are valid:

- `bin/FIRE7mp --variables 80_90+_100+_1 --config examples/doublebox3N --parallel`

Here 80 stays constant for the first variable, and second and third get shifted resulting in pairs $(90, 100), (91, 101), \dots, (105, 106)$.

- `bin/FIRE7mp --variables 80^1_90^1+_100^1+_1 --config examples/doublebox3N --parallel`

Here 80 stays constant for the first variable, and second and third variable exponents get shifted resulting in pairs $(90^1, 100^1), (90^2, 100^2), \dots, (90^{16}, 100^{16})$.

The resulting table files also have the plus symbol at the places specified in the variables setting, to indicate that IBP reduction results for multiple parameter values are stored in the same file. `FIRE7_MPI`, when using `FIRE7mp` with the `-M/-multitables` option, understands the above table file format during automated reconstruction. Please note that the number of values is not saved in the table filename, so in case FIRE is reconfigured for another value for the number of probe points, the old tables cannot be used.

2.6. Reduction and reconstruction of combinations

In some situations, for example when working with differential equations, one does not need reduction results for particular Feynman integrals, but for their linear combinations. It is also known that the master coefficients in the reduction results of these combinations can be much simpler than the ones for individual integrals involved in the combinations. To reduce a combination, one basically needs to reduce all integrals in it, so with the classical analytic IBP approach, working with combinations does not help much. However,

with the modular approach, the use of combinations becomes efficient. The reason is that we can run the reduction for individual integrals but save only tables for combinations. Thus the coefficients to be reconstructed become much simpler, resulting in fewer required probe points for the reconstruction.

The syntax for specifying combinations can be found in `examples/boxc.m`.

3. Improvements not related to the modular approach

There is a number of improvements for the operation of FIRE in general, not exclusively for the modular approach. This section summarizes the most important features. Some of them already appeared in the public version FIRE6.5, but the features themselves were not described at that moment, so we are presenting them here. Moreover, some of them were updated and improved after they appeared in the FIRE6.5 code.

3.1. *Split masters mode*

An alternative to the modular approach for the parallelization of IBP reduction is separating the reduction to different master integrals within the classical (rational polynomial) approach. To proceed this way one needs to determine first the list of master integrals, for example, by running a sample modular job for a single probe point. Afterwards there is a syntax to use FIRE7 so that only a subset of master integrals is considered to be non-zero, and other ones are directly set to zero in the system of IBP linear equations. This simplifies the reduction and reduces the consumption of RAM and CPU time.

The `#masters |first-last|filename` instruction in the configuration file enables this mode. Here `first` and `last` are numerical positions of master integrals listed in the following file (the indexing starts from 1 not 0). For example, `#masters |1-2|doublebox.masters` sets all master integrals, other than the 1st and 2nd ones listed in `doublebox.masters`, to be zero. Similarly, `#masters |4-4|doublebox.masters` retains only the 4th master integral. This allows parallel runs that each target a subset of master integrals. One should be sure that no more master integrals are going to appear, otherwise FIRE will crash immediately upon encountering one. The output table files will have a suffix with the master range `first-last` before the `.tables` extension. After all reductions (targeting different subsets of master integrals) are ready, one can combine them in *Mathematica* with the `CombineTables` function. The syntax is `CombineTables[filename, list]`,

for example, `CombineTables[filename, {{1, 2}, 3, 4}]`. The list can contain either numbers or pairs of numbers indicating start and end of a range.

It might be also useful to provide the list of masters, with the syntax `#masters filename`, even if the split mode with the extra `|first-last|` syntax is not used, as this knowledge can improve reduction efficiency.

For historical reasons, the `#masters` option also has another effect when it is used without any additional argument such as the filename. In this case, FIRE will run in a different mode which only finds the list of master integrals but does not perform the backward step of the IBP reduction.

3.2. Presolving IBP relations

There are two opposite approaches to working with IBP equations. The initial one, originating from Laporta is to generate all needed relations with symbolic indices (propagator/ISP power) and to solve the system after substituting specific numerical values of the indices. The other one aims to solve relations before substituting indices, i.e. keeping symbolic dependence on the indices, which was attempted with the use of Gröbner bases [67, 20, 22, 21], LiteRed [8, 9, 10] by Roman Lee, generating functions [25, 26], and variants of the syzygy equation method [28, 29]. However, in many cases trying to solve all relations without substituting indices fails or makes relations too complex.

The pre-solving of IBP relations is an attempt to somewhat combine these approaches and to allow the reduction program to combine IBP identities before substituting indices, but only linearly, that is, not applying shift operators to them. This reduces the complexity of the system without making the equations have too many terms.

The option is on by default and can be switched off with the option `#no_presolve` in the `config` file or reverted to the variant implemented in FIRE6.5 by `#old_presolve`. The old presolve variant takes IBPs as relations and performs forward Gaussian elimination of those, aiming on positive shifts in the chosen sector. The new full pre-solving also applies backward Gaussian elimination. We expect the full presolve to be efficient in most cases, but are aware of situations where one might wish to switch it off or use the old intermediate variant.

3.3. Orderings

As suggested in [17], ordering choices in solving the IBP linear system can have a great impact on performance. **FIRE** now allows users to set the ordering choices as a sequence of letters in the `config` file. The ordering is generated in each sector, taking absolute differences of integral indices and the sector corner (i.e. the index vector where positive powers are replaced by 1 and non-positive powers are replaced by zero). The ordering letters are applied in the order they appear.

The following letters are valid:

- **A**: 1 at each position, i.e. sum of all index shifts;
- **P**: sum of all positive index shifts;
- **N**: sum of all negative index shifts;
- **r**: modifier to next letter such as **a**, **p**, **n** indicating that the ordering part will be reverse (from last to first);
- **i**: modifier to next letter such as **a**, **p**, **n** indicating that the ordering part will be inverse lexicographic, i.e. swapping “less than” and “greater than” of lexicographic ordering;
- **a**: lexicographic ordering over all indices;
- **p**: lexicographic ordering over positive indices;
- **n**: lexicographic ordering over negative indices;
- **l**: special ordering originating from papers from Lee [23].

A use of a proper ordering setting can have a significant impact on performance.

3.4. Preferred integrals

While this is not exactly a new feature, it requires an explanation since it might often confuse users. **FIRE** automatically detects master integrals as those integrals in sectors that cannot be reduced to lower sectors or lower integrals in the same sector under the chosen ordering. Sometimes one needs to influence the choice, and this can be done with the preferred integral list

and the `pos_pref` setting. Note, that in case a sector has integrals provided with `#preferred`, then the `pos_pref` has no impact on that sector.

The preferred integral list makes **FIRE** mark some integrals in a sector with a preferred bit (in addition to data fields storing the indices in the C++ data structure representing the integrals), making them lower than all unmarked integrals in the same sector (under the comparison function used in the code). This however does not guarantee that they are going to become master integrals, if only because one might specify more preferred integrals in the sector than there are masters in it. Unlike the `master` setting, specifying extra preferred integrals does not cause the reduction to crash. All integrals listed as preferred in a sector are compared between each other under the chosen ordering, which determines which subset will be made master integrals. It is also important to mention that all integrals listed as masters and all integrals on the left-hand sides, and right-hand sides of provided rules using the `rules` setting are also treated as preferred.

Another option related to preferred integrals is the `pos_pref` option of the configuration file. By default, it is equal to 1. This option is applied to sectors that have no manually listed preferred integrals and makes all integrals with up to `pos_pref` dots preferred, meaning by default **FIRE** prefers integrals with 1 dot as masters to integrals with 1 irreducible numerator no matter what ordering is chosen. This is the typical setting when the user tries to choose masters with dots. However in some sectors there might be many masters, and there are not enough integrals with 1 dot for this, so the `pos_pref` can be increased to ensure that integrals with dots, rather than numerators, are chosen as masters.

Also, `pos_pref` can be made negative to prefer masters with numerators up to a rank equal to the absolute value of `pos_pref`.

Another use of the `pos_pref` is together with **LiteRed** — **FIRE** can use internal sectors symmetries of **LiteRed** and applies them to preferred integrals. The reason to not apply them to all integrals is that this turns out to slow down the reduction.

4. Installation and usage

In this section we aim to provide a guide on how to install and use **FIRE** listing important options. The previous manual [14] was published more than five years ago and definitely needs an update.

4.1. Installation

To install FIRE7 one should first clone it from the gitlab repository:

```
git clone https://gitlab.srcc.msu.ru/feynmanintegrals/fire.git
```

One can also add one more argument to the clone command specifying the folder (`fire` by default).

Note that there is a set of required common libraries and compiler tools not shipped with FIRE. The most common set on a Debian-based distribution is²

```
apt-get install g++ make cmake automake \
autoconf zlib1g-dev libcln-dev
```

Other commands should be performed in the `fire/FIRE7` folder of the newly cloned repository. The first step is to launch the `configure` script by `./configure` providing options if needed. There are the following options:

- `--enable-debug` — enables debugging symbols and stack trace printing in case of crashes. Has minimal impact on performance;
- `--mprime=value` — sets another number of probe point used in `FIRE7mp`; please note that tables generated for different values are incompatible;
- `--enable-zlib`, `--enable-zstd`, `--enable-snappy` — enable compressors which can be set in `config` files;
- `--cpp=value`, `--cc=value` — set `c++` and `c` compilers, defaults are `g++` and `gcc`;
- `--enable-tcmalloc`, `--enable-mimalloc` — two options to choose another memory allocator (incompatible with each other);
- `--build-openmpi` — to build the openmpi compiler in case the one on the cluster is too old;
- `--prebuilt-mpfr`, `--prebuilt-gmp`, `--prebuilt-flint`, and `--prebuilt-zstd` — use a prebuilt version of listed libraries; this shorten compile times but might be incompatible with the system in use;

²`libcln-dev` can be omitted unless one wants to use the non-default `ginac` backend for simplifying rational functions.

- `--enable-ginac`, `--enable-pari`, `--enable-symbolica` — used to enable alternative simplifying libraries (by default FIRE comes with FLINT and Fermat); note that Symbolica [68] needs a valid license;
- `--fermat7` — use version 7 instead of 5 of Fermat;
- `--enable-lto` — enable gcc link time optimization for the prime version. This can result in a 5 percent speedup, but spoils debugging and crash diagnostics;
- `--small-point`, `--many-sectors[=value]`, `--large-point` — these options influence how large the point size (particular integral) is, the maximal number of indices and how many non-zero sectors one can have; the possible variants are shown in Table 4.1.

Options	Point size	Max. shift	Max. value	Sectors
<code>--small-point</code>	16	18	31	2^{15}
default	24	22	127	2^{15}
<code>--many-sectors</code>	24	20	127	2^{20}
<code>--many-sectors=n</code>	24	20	127	2^n
<code>--large_point</code>	32	28	127	2^{20}
<code>--large_point</code> and <code>--many-sectors=n</code>	32	28	127	2^n

Table 1: Different point combinations

The point size might be important, especially in the modular case, since the coefficients stored use only 8 bytes, so the point size might greatly influence the amount of RAM needed. However the significance is decreased in the multiprime mode. Max. shift stands for the maximal total of the absolute differences of indices of integrals involved and corresponding sector corners. For example, the shift of $(1, 3, 5, -2)$ is $(1-1)+(3-1)+(5-1)+(-(-2)) = 8$. For diagrams up to 5 loops, 22 indices should normally be sufficient, while for 6 loops, one should switch to a larger point size. Be careful with the small point setting to be sure that the total shift of indices does not exceed a relatively small number (max. shift). The “Sectors” column of the table stands for the number of non-zero sectors; with the `--many-sectors` option, 4 bytes are used to store a sector number, but do not blindly increase the setting, as

there's a proportional amount of RAM needed to store sector numbers and some pointer, so with `--many-sectors=31` around 12 GB will be used only for this.

An attentive user might also notice the `--enable-fire7np` which enables private modular reduction improvements as a submodule.

After running the `configure` script one gets its options saved in the `previous_options` file and creates a `Makefile`. Now one can build the libraries FIRE depends all with the `make dep` command. The more options enabling libraries above were set, the more libraries are going to be built at this point. As usual, `make dep -j` or `make dep -j n` for the number of threads *n* will enable building libraries in parallel. However, there is a known issue that some of the libraries on some filesystems refuse to build in parallel if they cannot lock some file, so a non-parallel build has to be used.

After the libraries are built properly, one can build FIRE with `make -j`. If the MPI part is needed, also `make mpi` should be called. The build can be verified with `make test`.

An alternative way to build FIRE is to use Docker, which simplifies the installation process and runs FIRE in a self-contained sandbox. The FIRE image can be obtained from Docker Hub with the command

```
docker pull asmirnov80/fire
```

Now one can launch FIRE from the container like

```
docker run --rm fire7 /app/FIRE7/bin/FIRE7 --help
```

As Docker containers are sandboxed, any host folder needs to be explicitly mounted as a volume to be accessed by FIRE inside the container. For example, the following command mounts the current directory as the `/host` folder in the container and makes it the working directory when launching FIRE:

```
docker run --rm -v ${PWD}:/host -w /host fire7 \
/app/FIRE7/bin/FIRE7 --help
```

For advanced users who want to re-build the Docker image after local modification to the FIRE code, here are the instructions. First, make a new clean clone of FIRE, change directory to the outer fire folder and run

```
docker build -t fire7 .
```

to build the image.³ Currently the build uses the

³ *Warning:* trying to build a docker image within a folder where FIRE has been already

https://hub.docker.com/repository/docker/asmirnov80/for_fire/ image with preinstalled external libraries as a builder image to bootstrap the final image. The builder image in turn uses the `debian:trixie-slim` base image.

4.2. Configuration file settings

Since the **Mathematica** part of **FIRE** remains almost unchanged since previous versions, we do not repeat those instructions and instead refer to the previous manual. On the other hand, the **c++** part has changed a lot, so here we will list the possible commands in the `config` file, main file when launching the **c++** reduction. As a reminder, having a `config` file with the name `example.config`, one launches **FIRE** with `bin/FIRE7 -c example`, as the `.config` extension is added automatically. The configuration file supports a number of instructions, each of those starting from the symbol `#`. Double `##` is a commented line and has no effect. The order of some commands is important, but the current version is supposed to stop immediately in case of an incorrect order.

The first group of commands should be placed before the `#start` directive:

- `#calc`: sets the simplifier library, default is `flint` (in previous versions, the default was `fermat`);
- `#ordering`: an important setting specifying an ordering type, for details see section 3.3; the default value according to the notation in that section is `AN1`;
- `#index_ordering`: specifies order of indices in the ordering, by default from first to last;
- `#positive`: a way to provide comma-separated indices that have to be positive, i.e. IBP on a cut (see e.g. Ref. [33]); a fast way to perform partial reduction;
- `#compressor`: sets compressor for coefficients, default is `lz4`, can be switched off with `none`;

built might lead to build errors or unexpected behavior. Each time this command is issued, the contents of the **FIRE7** folder is copied into the container and rebuilt.

- **#threads**: sets the number of threads working in parallel, each thread in its own sector (of same level);
- **#sthreads**: overrides the threads setting at substitution stage;
- **#fthreads**: sets the number of simplifier library threads shared by evaluation threads. If the number is preceded by the **s** letter, then each thread has the specified number of own simplifier threads;
- **#variables**: lists comma-separated variables that exist in expressions; variables can be followed with **->** and a value;
- **#database**: provides a path to temporary database files;
- **#storage**: provide a path to another folder where database files are copied so they are not broken in case of job interrupts; useful to be able to continue a job from an intermediate point;
- **#forward** and **#backward**: run only forward (first stage) or backward (second stage) run of **FIRE**; needs the storage to be used;
- **#bucket**: fine-tuning of kyotocabinet database;
- **#wrap**: makes databases wrapped into a single file; might be useful in order to decrease the number of files on disk;
- **#clean_databases**: cleans up temporary databases after completing the reduction;
- **#prime**: for the modular version, specifies the prime number index (but it is more advisable to provide it from command line);
- **#allIBP**: makes all IBPs used, switches off the Lee ideas [23] to remove some of those;
- **#no_presolve** and **#old_presolve**: see section 3.2;
- **#even** and **#odd**: make **FIRE** consider all integrals of other parity equal to zero;
- **#pos_pref**: see the 3.4 section.

After options listed above the `#start` directive should appear, followed with instructions related to file paths of the diagram in use:

- `#folder`: provides a path to other files so that full path can be written in one place;
- `#hint`: an experimental setting providing a path to a folder where “hint” files are saved. On consequent run with access to the hint files might use less time knowing which IBP relations are to be used. This might make sense to create hint files with a modular run before a polynomial reduction. However this option has not been tested well and we are currently unable to provide proper benchmarks and confirm that it is useful. Please use it at your own risk;
- `#rules`: path to a file with rules — mappings of individual integrals to linear combinations of other integrals; rules cannot be recursive;
- `#lbases`: path to a file with rules originating from `LiteRed`;
- `#output`: path to output file with tables;
- `#preferred`: path to a file specifying preferred master integrals;
- `#integrals`: path to input file with integrals.

4.3. Command-line options of *FIRE*

The main `FIRE` binaries (`bin/FIRE7`, `bin/FIRE7p`, `bin/FIRE7mp`) also accept some of the options from the command line. The explanations are available if calling `bin/FIRE7 --help`, most of the options correspond to the configuration file directives. Let us outline important options that do not have direct analogues in the configuration file:

- `--config <value>`: the main obligatory option to provide path to the configuration file;
- `--parallel`: set this option in case multiple `FIRE` instances are running at the same time, so that they do not conflict on temporary files;
- `--quiet`, `--QUIET`: suppress most of output, the second one is more effective;

- `--calc_options <value>`: comma-separated options to be passed to calc library with `fuel::setOption`; might be usefull for testing things such as `Symbolica` factorization of denominators;
- `--variables <value>`: set variable values, see the 2.2 section;
- `--large_variables`: indicates that variables passed can be large and require an external library to be handled;
- `--folders <value>`: indicates how tables should be saved in subfolders bases on the quotient of the largest index shift in passed variables and the value;

4.4. Reconstruction tool and MPI parallelization

The reconstruction binary has the following syntax: `bin/reconstruct [options] filename range`. Here `filename` should be the target reconstruction table which should follow the pattern where in the end of the filename the variable names or their values follow underscore-separated. The last number after the underscore stands for the prime number index or zero for the final reconstruction to rational numbers. Let us provide one example to demonstrate the syntax:

```
bin/reconstruct --prime 1 --method thiele \
--reconstruction_variable d_100 \
tests/outputs/doublebox_d_7_3_1.tables 13
```

Here a Thiele reconstruction by `d` (for reconstruction methods see section 2.3) is performed, taking 100 as the initial value of `d` and using up to 13 tables. This is a double box example, where `s` and `t` have values 7 and 3, and the first large prime number is used as the modulus. The reconstruction tool will read the tables from disk with the same name as the target table with different values of `substituted` from 100 to 112. If some of the tables are missing, it will try to reconstruct with existing ones (however for the Zippel method missing tables are not allowed due to the algorithm nature).

Calling `bin/reconstruct --help` prints help on all the options of the reconstruction tool. In most cases the used is not supposed to call this tool directly for it is called by the MPI wrapper (and the call syntax is seen in the log), so we are not going into more details with this binary.

The MPI wrapper `bin/FIRE7_MPI` should be called to start the automatic reduction and reconstruction process. Let us demonstrate the Zippel

approach options on a command provided in examples and comment all options:

```
mpirun -np 4 bin/FIRE7_MPI  
-Z -r -E -S -D -B -P 10 -c examples/doubleboxN
```

- `mpirun -np 4 bin/FIRE7_MPI` — launches 4 instances;
- `-Z` or `-Zippel` — uses the Zippel approach;
- `-r` or `-reconstruct` — requests automatic reconstruction;
- `-E` or `-early_abortion` — make FIRE immediately stop with an abort in case at some point it detects that with the current limits it cannot reconstruct something;
- `-S` or `-last_separated` — instructs that last variable is separated when factorizing denominators;
- `-D` or `-delete_tables` — tells FIRE to delete intermediate tables;
- `-B` or `-big_primes` — makes FIRE use big primes as starting values;
- `-P 10` or `-rational_reconstruction_limit 10` — limits to 10 primes.

This is enough for a quick start. Other options can be seen in the `bin/FIRE7_MPI -help` command and are mostly related to fine-tuning performance on clusters. We will be happy to assist with such tuning for large-scale problems.

4.5. New tools for manipulating IBP tables

We have also developed a number of useful utilities working with FIRE tables. All of them produce the list of options if called with the `-help` flag, so let us outline briefly their usage. Some practical examples will be given in Section 5.1.

- `combine` — the `combine` utility shipped with FIRE is used to apply IBP tables to linear combinations of integrals and takes three filenames as arguments. It takes a linear combination of integrals from the first file, uses IBP tables from the file in the second file to replace each integral by the reduction result, combines the reduction results with the respective prefactors, and saves the result to an output file. This can be used to e.g. reduce a complete amplitude which is a sum of integrals.

- **diff** — the **diff** utility shipped with FIRE7 accepts two arguments and compares two files in FIRE’s table format. The reason to use **bin/diff** instead of a system **diff** is that tables can be equivalent in an algebraic sense but not identical bitwise. Note that is important to specify the list of variables. The utility returns no output if the two tables agree. For non-coinciding tables, though, a non-zero error code is returned (useful for Unix pipes and scripts).
- **add** — accepts two input tables and one output file as arguments. The reduction results of each integral from the two tables is added to be the new reduction result in the output table. The subtraction mode can be turned on with the **-s** command line flag to compute the difference of reduction results instead. This is more flexible than **diff** for two reasons. First, an actual difference table is produced, though an additional zero/nonzero boolean conclusion can also be printed out if one passes the **-verbose** or **-V** command line option. Second, The input tables can be produced with two “incompatible” configuration files, which may cause **diff** to fail.
- **tables2rules** — converts FIRE tables format into **Mathematica** set of rules;
- **substitute** — used to take one table, substitute a number of variables into it and save in as another table file; The output filename is generated automatically based on the requested substitutions.

Alternatively, one can pass a configuration file argument with **-c** and omit the table filename. The filename will be deduced from the configuration file, and substitutions must be specified for all variables. The specified values will be prepended before **.tables** in output filenames. This utility is most useful for testing purposes;.

4.6. Usage with *LiteRed*

Let us start this small section with a quick notice that although the package **LiteRed** [8, 9, 10] is shipped with **FIRE**, this is a standalone package by another author and should be cited separately if used. We keep shipping **Litered 1.8** but suppose that **FIRE7** can work with **LiteRed2** that should be downloaded and installed independently.

The way **FIRE** can be used with **LiteRed** requires **Wolfram Mathematica**. A basic setup is the following:

- Create a `start` file, save it, unload the `Mathematica` kernel.
- Run a generation of `LiteRed` data related to the diagram in use. In general case we recommend using the generation of `LiteRed` detection of zero sectors and of symmetries, external (between different sectors) and also internal (inside sectors). Unload the `Mathematica` kernel.
- Run the part of `FIRE` that transforms the information from `LiteRed` folders to an `lbases` file that can be referred from the `config` setup for the `c++` reduction. At the same time do not forget to save a new `start/sbases` file as it stores the information on which sectors are mapped to other sectors by means of the `LiteRed` symmetries.

Now one can perform the `c++` reduction with the use of `LiteRed` symmetries. For the syntax details please refer to the `FIRE6` paper [14].

5. Examples and benchmarks

5.1. Examples for IBP table tools

We present a mini-tutorial based on one-loop box integrals, to illustrate the use of the new tools introduced in Section 4.5, but we will start by reminding users about basic usage of `FIRE`. The various files mentioned can be found inside the `examples` folder.

5.1.1. basic usage

We define the integral family in `Mathematica` and export a `start` file, as in `examples/box.nb`:

```
SetDirectory[NotebookDirectory[] <> "../"];
Get["FIRE7.m"];
Internal = {k};
External = {p1, p2, p4};
Propagators = {-k^2, -(k+p1)^2, -(k+p1+p2)^2,
               -(k+p1+p2+p4)^2};
Replacements = {p1^2->0, p2^2->0, p4^2->0,
                p1 p2->-S/2, p2 p4->-T/2,
                p1 p4->(S + T)/2, S->1, T->1};
PrepareIBP[];
Prepare[AutoDetectRestrictions->True, LI->True];
SaveStart["examples/box"];
```


This creates the file `box.start`, which contains information about the integral family in a format that can be read by the C++ version of FIRE used in the terminal. We would like to reduce the integrals

$$G_1(2, 2, 1, 1), \quad G_1(2, 1, 1, 1) \tag{1}$$

to master integrals. We have arbitrarily assigned the “problem number” to be 1, and therefore wrote G_1 above. The integrals are written to `examples/box.m` with the content:

```
{
  {1,{2,2,1,1}},
  {1,{2,1,1,1}}
}
```

The two integrals are shown as lists of indices, preceded by the problem number 1. The configuration file is `examples/box.config`:

```
#compressor      none
#threads         1
#fthreads        4
#variables        d
#start
#folder           examples/
#problem          1 box.start
#integrals        box.m
#output           ../tests/outputs/box.tables
```

We run the shell command (from the directory `fire/FIRE7`, as usual)

```
bin/FIRE7 -c examples/box
```

which produces the IBP reduction table `tests/outputs/box.tables`.

5.1.2. *converting tables to Mathematica format*

In previous versions of FIRE, the only way to load an IBP table into Mathematica is through the `LoadTables` function provided by FIRE’s Mathematica package. FIRE 7 introduces a new tool, `tables2rules`, that directly works on the command line without running Mathematica. Running

```
bin/tables2rules tests/outputs/box.tables \
  tests/outputs/box.tables.m
```

converts the table to Mathematica format and write to the latter file specified in the command line.

5.1.3. divide and conquer: split master mode

Complicated IBP reduction tasks can be divided into smaller jobs each keeping only a subset of master integrals. We illustrate the usage with the box integrals. First, we save the list of master integrals,

$$G_1(1, 1, 1, 1), \quad G_1(1, 0, 1, 0), \quad G_1(0, 1, 0, 1) \quad (2)$$

to the file `examples/box.masters`:

```
{
  {1,{1,1,1,1}},
  {1,{1,0,1,0}},
  {1,{0,1,0,1}}
}
```

Then we create a configuration file `box_split.config` for IBP reduction targeting only the first master integral, using the `#masters` syntax explained in Section 3.1:

```
#compressor      none
#threads         1
#fthreads        4
#variables       d
#start
#folder          examples/
#problem         1 box.start
#masters         |1-1|box.masters
#integrals       box.m
#output          ../tests/outputs/box.tables
```

Similarly, we create another configuration file `box_split_part2.config` targeting the remaining master integrals, i.e. the 2nd to 3rd ones:

```
#compressor      none
#threads         1
#fthreads        4
#variables       d
#start
```

```
#folder          examples/  
#problem         1 box.start  
#masters         |2-3|box.masters  
#integrals       box.m  
#output          ../tests/outputs/box.tables
```

Running the shell commands

```
bin/FIRE7 -c examples/box_split  
bin/FIRE7 -c examples/box_split_part2
```

produces the two tables:

`tests/outputs/box.1-1.tables` and `tests/outputs/box.2-3.tables`, which only include the 1st master integral and the 2nd-3rd master integrals, respectively. Note that the output file names specified by the configuration files have been automatically modified to include the extra information about the range of the master integrals included.

5.1.4. add tables from split master mode

We combine the above two tables with `bin/add`, specifying “d” as the only variable involved in the tables.

```
bin/add -v d tests/outputs/box.1-1.tables \  
        tests/outputs/box.2-3.tables \  
        tests/outputs/box.added.tables
```

The last argument above is the output file. To check agreement with the table obtained previously without the split-master mode, we compute their difference with the “subtraction” mode of `bin/add`:

```
bin/add -v d -sV tests/outputs/box.added.tables \  
        tests/outputs/box.tables /dev/null
```

We have discarded the output by directing it to `/dev/null`, while using the “verbose” `-V` flag to print out a conclusion,

```
Zero table written
```

which confirms that the two tables are consistent, as desired.

5.1.5. `combine` to reduce linear combinations of integrals

Suppose we would like to reduce a linear combination of the integrals in Eq. (1),

$$(d+2)G_1(2,2,1,1) + d^2G_1(2,1,1,1). \quad (3)$$

We create a file, `examples/box.combinations`:

```
{
  {G[1,{2,2,1,1}], "d+2"},
  {G[1,{2,1,1,1}], "d^2"}
}
```

Then we run `bin/combine` to reduce the linear combination against the existing IBP table, `tests/outputs/box.tables`,

```
bin/combine -v d examples/box.combinations \
  tests/outputs/box.tables \
  tests/outputs/box.reduced
```

We have again specified d as the only variable, and the output has been written to `tests/outputs/box.reduced`. The output file contains, in Mathematica notation, the coefficients for each of the three master integrals, and the content should be self-explanatory.

5.1.6. Streamlined direct reduction of linear combinations

If one wants to reduce a linear combination of a large number of integrals, reducing all these integrals in the above workflow will result in a large table, even though the final result may be rather short. To bypass the need for large intermediate files, we can directly reduce linear combination of integrals. To do this for the simple example Eq. (3), we create an integral combination file `examples/boxc.m`,

```
{
  {
    {d+2, {1,{2,2,1,1}}},
    {d^2, {1,{2,1,1,1}}}
  }
}
```

More generally, the file content can be a list of linear combinations and also ordinary single integrals—the file above is a list of only one combination, which explains why the file has one more nested level of brackets than `box.combinations`.

Then we create a new configuration file `examples/boxc.config` which differs from `examples/box.config` only in specifying `boxc.m` for the `#integrals` option and `boxc.tables` as the output. Running

```
bin/FIRE7 -c examples/boxc
```

produces the output file. Inspecting the file, e.g. by converting it to a more readable Mathematica format with

```
bin/tables2rules tests/outputs/boxc.tables \
    tests/outputs/boxc.tables.m
```

shows that the linear combination of integrals is reduced to master integrals.

5.1.7. *substitute numerical values for variables in tables*

Sometimes, it is necessary to turn an analytic IBP table into a numerical one before further use. For example, the following command substitutes an arbitrary value of 53 for the d variable, modulo the 1st large prime tabulated for use with FIRE (see Section 2.2):

```
bin/substitute -c examples/box -v 53_1
```

With the above command, the `substitute` tool reads the configuration file `examples/box.config` to deduces that the analytic IBP table is located at `tests/outputs/box.tables`, then performs the requested substitutions and saves the result to `tests/outputs/box_53_1.tables`. Generally, when multiple variables are involved, their numerical values need to be all given in the command line, separated by underscore characters, with the last number n specifying the modulus as the n -th FIRE prime.

5.1.8. *Verifying agreement with direct numerical run*

Let us check that the above numerical substitution result agrees with a table produced by a direct numerical IBP run. First, let us back up the substitution result,

```
cp tests/outputs/box_53_1.tables \
    tests/outputs/box_53_1.tables.bak
```

Then we use `FIRE7p` to perform the numerical IBP run at $d = 53$ modulo the 1st FIRE prime,

```
bin/FIRE7p -v 53_1 -c examples/box
```

Let us verify that the newly created (overwritten) `box_53_1.tables` agrees with the backup of the previous file:

```
bin/diff -v d tests/outputs/box_53_1.tables \
    tests/outputs/box_53_1.tables.bak
```

The two files agree, as `bin/diff` exited without output.⁴

5.2. Benchmarks for modular IBP reduction

We provide benchmarks comparing the modular arithmetic performance of FIRE 6.5 and FIRE 7 with different orderings (default, Aia, Apn), as explained in Section 3.3, for the IBP reduction of the following integrals, setting all kinematic variables to random numerical values and performing the IBP reduction modulo a 64-bit prime number. The multi-threading settings `#threads` and `#fthreads` (as explained in Section 4.2) are both 4 in the runs.

- The nonplanar double box diagram with four massive legs, three of which share the same mass, and massless internal lines, with $p_1^2 = p_2^2 = p_3^2 \neq p_4^2$, in Fig. 1, with two tensor integrals each with 8 ISP powers, $[(k_1 + p_1)^2]^4 [(k_2 - p_4)^2]^4$ and $[(k_1 + p_1)^2]^8$.
- The nonplanar double pentagon diagram Fig. 2, with the tensor integral with 4 ISP powers, $(l_2 + p_5)^2 (l_1 + p_4)^2 [(l_1 + p_2 + p_4)^2]^2$.
- The soft-expanded 4-loop quadruple box diagram Fig. 3 for the scattering of two massive particles with a soft momentum transfer in the t-channel, with the tensor integral with 3 ISP powers, $(k_2 \cdot k_4)(k_3 \cdot k_4)^2$. This diagram is taken from a 4-loop calculation of the classical limit of charged particle scattering in electrodynamics [69].

The benchmark results, including run times and estimated memory usage, are in Table 2. The machine used has the Intel Xeon Platinum 8280 CPU

⁴As covered previously, in more general situations where two tables are produced from incompatible configuration file settings, e.g. different split-master ranges, a more flexible (but more resource-intensive) alternative to `bin/diff` is `bin/add -sV`.

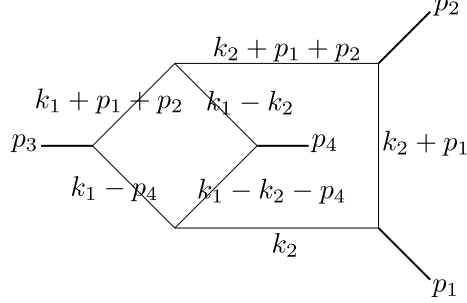


Figure 1: The nonplanar double box diagram with massive legs. The legs p_1 , p_2 and p_3 share the same mass, while p_4 has a different mass. The list of seven propagator denominators and two ISPs are $(k_1 - k_2 - p_4)^2$, $(k_1 + p_1 + p_2)^2$, $(k_1 - p_4)^2$, $(k_1 - k_2)^2$, k_2^2 , $(k_2 + p_1)^2$, $(k_2 + p_1 + p_2)^2$, $(k_1 + p_1)^2$, and $(k_2 - p_4)^2$.

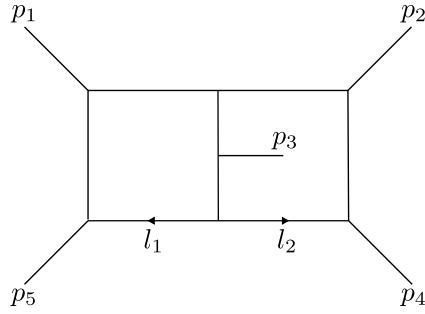


Figure 2: The nonplanar double pentagon diagram. The list of eight propagator denominators and three ISPs are l_1^2 , $(l_1 - p_5)^2$, $(l_1 - p_5 - p_1)^2$, $(l_2 - p_4 - p_2)^2$, $(l_2 - p_4)^2$, $(l_2)^2$, $(l_1 + l_2)^2$, $(l_1 + l_2 + p_3)^2$, $(l_2 + p_5)^2$, $(l_1 + p_4)^2$, and $(l_1 + p_4 + p_2)^2$.

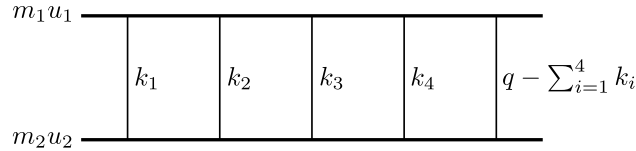


Figure 3: The soft-expanded 4-loop quadruple box diagram. The massless vertical lines in the middle are considered as soft. The expansion linearizes the massive propagators shown as horizontal lines. The massive momenta running through the top horizontal line and bottom horizontal line are approximately $m_1 u_1$ and $m_2 u_2$, respectively, where u_1 and u_2 are normalized velocities. The thirteen propagators are nine ISPs are $-2k_1 \cdot u_2$, $-2(k_1 + k_2) \cdot u_2$, $-2(k_1 + k_2 + k_3) \cdot u_2$, $-2(k_1 + k_2 + k_3 + k_4) \cdot u_2$, $2k_1 \cdot u_1$, $2(k_1 + k_2) \cdot u_1$, $2(k_1 + k_2 + k_3) \cdot u_1$, $2(k_1 + k_2 + k_3 + k_4) \cdot u_1$, k_1^2 , k_2^2 , k_3^2 , k_4^2 , $(q - k_1 - k_2 - k_3 - k_4)^2$, $k_1 \cdot q$, $k_2 \cdot q$, $k_3 \cdot q$, $k_1 \cdot k_2$, $k_1 \cdot k_3$, $k_1 \cdot k_4$, $k_2 \cdot k_3$, $k_2 \cdot k_4$, and $k_3 \cdot k_4$.

and 192 GB of DDR4 RAM. We can see that FIRE 7, under any of the orderings chosen, outperforms FIRE 6.5 under the default ordering. The results for the fastest ordering for FIRE 7 are highlighted in red, while FIRE 6.5 results are highlighted in blue, to clearly highlight the ordering choice with the largest time savings. Furthermore, we test the multiprime mode, explained in Section 2.5, to perform IBP reduction for 16 numerical probes simultaneously while reusing the best ordering choice found for each integral family.⁵ The *average* run time per numerical probe and the memory usage is shown in the last column of Table 2. Compared with FIRE 6.5 with default orderings, the multiprime mode saves the run time by a factor of about 5 to 50 depending on the integral family, though sometimes with higher memory usage.

Integral family	FIRE 6.5	default	Aia	Apn	Multiprime
Double box with massive legs	52s, 0.45G	43s, 0.46G	45s, 0.43G	39s, 0.39G	11s, 1.4G
nonplanar double pentagon	100s, 0.38G	92s, 0.41G	60s, 0.38G	78s, 0.40G	20s, 1.6G
soft-expanded 4-loop box	237s, 6.4G	151s, 6.9G	30s, 3.7G	152s, 6.4G	4.7s, 4.1G

Table 2: Benchmark results for three integral families, as run times in seconds and estimated memory usage in Gigabytes, with the modular arithmetic mode of FIRE. Each benchmark is run three times, with the shortest time taken to reduce noise from background processes.

5.3. Benchmark for analytic IBP reduction

We also present a benchmark for analytic IBP reduction for a 3-loop integral family with massive internal lines. The kinematics is exact forward scattering with massless incoming momenta, $q_1 + q_2 \rightarrow q_1 + q_2$. The diagram is shown in Fig 4. We reduce only one sample integral in this benchmark, where the 4th propagator $(l_3 - q_2)^2 - m^2$ is raised to a 2nd power, the 8th propagator is absent (canceled), and the numerator is the product of the two ISPs, $(l_1 + l_2 + l_3)(l_1 - q_1)^2$. FLINT (default for FIRE 7) is used as the simplifier for all the runs. The performance of FIRE 6.5 and FIRE 7 with

⁵Note that the ordering is also affected by how the propagators and ISPs are ordered by the user, and this information is given in the figure captions of the diagrams.

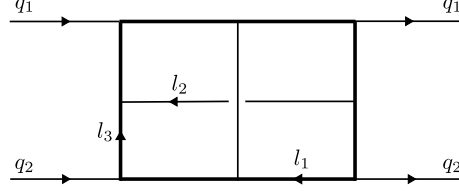


Figure 4: The three-loop forward-scattering diagram with massive internal lines, with $q_1^2 = q_2^2 = 0$, $(q_1 + q_2)^2 = s$. The list of 10 propagators are $l_3^2 - m^2$, $(l_2 + l_3)^2 - m^2$, $l_1^2 - m^2$, $(l_3 - q_2)^2 - m^2$, $(l_2 + l_3 + q_1)^2 - m^2$, $(l_1 + q_2)^2 - m^2$, $(l_1 + l_2 + q_1 + q_2)^2 - m^2$, $(l_1 + l_2 + q_2)^2 - m^2$, l_2^2 , and $(l_1 - l_3 + q_2)^2$. The two ISPs are $(l_1 + l_2 + l_3)^2$ and $(l_1 - q_1)^2$.

default orderings, as well as FIRE 7 with Aia and Apn orderings, are shown in Table 3. The various settings for running the benchmarks are identical to Section 5.2, except that `#threads` and `#fthreads` are now set to 24.

FIRE 6.5	default	Aia	Apn
11.6×10^3 s, 142G	4.7×10^3 s, 67 G	5.4×10^3 s, 78G	12.1×10^3 s, 211G

Table 3: Benchmark results for the three-loop forward scattering diagram with massive internal lines, as run times in seconds and estimated memory usage in Gigabytes, with the analytic mode of FIRE.

This time, FIRE 7 with the default ordering (also dependent on the order in which we supplied the list of propagators and ISPs) achieved the best performance. The results here are complementary to the ones for modular reduction in Section 5.2, and demonstrate the advantage of FIRE 7 for larger-scale analytic reduction runs.

6. Conclusion

We have presented the new version 7 of FIRE, a program for integration-by-parts reduction of Feynman integrals, aimed at cutting-edge calculations in perturbative quantum field theory. A main change is enhanced performance for IBP reduction with modular arithmetic, achieved by an improved presolve of IBP identities before replacing symbolic indices (propagator powers) by integer numerical indices. The new presolve involves Gaussian elimination with both forward and backward stages, and reduces the complexity of the IBP identities before proceeding with a conventional Laporta-style seeding of IBP equations. The user can also adjust the ordering of integrals in the implementation of the Laporta algorithm, which can have significant

impacts on run times and memory usage, as shown in the benchmarks. Note that experimenting with ordering settings is particularly suited to the modular approach, since one can find the best ordering setting using IBP runs at a small number of numerical probe points, and the performance characteristics will remain unchanged at a large number of new probe points needed to complete the reconstruction of analytic results.

The multiprime mode offers another significant performance enhancement, by performing IBP reduction at multiple probe points simultaneously to reduce the overhead incurred per point. In certain cases, combining a suitable ordering choice and the multiprime mode has resulted in a 50-fold speedup over the previous version of FIRE.

The usability of modular IBP reduction is greatly improved by the complete automation of MPI-parallelized reduction and subsequent reconstruction of coefficients. The performance of analytic IBP reduction is also enhanced, benefiting from improvements such as the updated presolve. Several new command-line tools are introduced for efficiently manipulating IBP tables, improving the user experience beyond the core IBP reduction tasks.

This release brings most of the current code of FIRE for public usage. An experimental implementation of improved seeding [70, 71, 43, 62, 72, 73, 74] remains in development and has been applied in Ref. [75], to be released in the future.

7. Acknowledgments

The work of AS was supported by the Ministry of Education and Science of the Russian Federation as part of the program of the Moscow Center for Fundamental and Applied Mathematics under Agreement No. 075-15-2025-345 (in part of developing reconstruction algorithms). The study of AS was conducted under the state assignment of Lomonosov Moscow State University (in part of optimizing reduction within the polynomial approach). M.Z.'s work is supported in part by the U.K. Royal Society through Grant URF\R1\20109. For the purpose of open access, the authors have applied a Creative Commons Attribution (CC BY) license to any Author Accepted Manuscript version arising from this submission. The authors acknowledge the Texas Advanced Computing Center (TACC) at the University of Texas at Austin for providing high-performance computing resources that have contributed to the research results reported within this paper. The research

was carried out using the equipment of the shared research facilities of HPC computing resources at Lomonosov Moscow State University [76].

References

- [1] K. G. Chetyrkin, F. V. Tkachov, Integration by Parts: The Algorithm to Calculate beta Functions in 4 Loops, Nucl. Phys. B 192 (1981) 159–204. doi:10.1016/0550-3213(81)90199-1.
- [2] F. V. Tkachov, A theorem on analytical calculability of 4-loop renormalization group functions, Phys. Lett. B 100 (1981) 65–68. doi:10.1016/0370-2693(81)90288-4.
- [3] T. Gehrmann, E. Remiddi, Differential equations for two loop four point functions, Nucl. Phys. B 580 (2000) 485–518. doi:10.1016/S0550-3213(00)00223-6. arXiv:hep-ph/9912329.
- [4] S. Laporta, High precision calculation of multiloop Feynman integrals by difference equations, Int. J. Mod. Phys. A 15 (2000) 5087–5159. doi:10.1142/S0217751X00002159. arXiv:hep-ph/0102033.
- [5] C. Anastasiou, A. Lazopoulos, Automatic integral reduction for higher order perturbative calculations, JHEP 07 (2004) 046. doi:10.1088/1126-6708/2004/07/046. arXiv:hep-ph/0404258.
- [6] C. Studerus, Reduze-Feynman Integral Reduction in C++, Comput. Phys. Commun. 181 (2010) 1293–1300. doi:10.1016/j.cpc.2010.03.012. arXiv:0912.2546.
- [7] A. von Manteuffel, C. Studerus, Reduze 2 - Distributed Feynman Integral Reduction (2012). arXiv:1201.4330.
- [8] R. N. Lee, Presenting LiteRed: a tool for the Loop InTEgrals REDuction (2012). arXiv:1212.2685.
- [9] R. N. Lee, LiteRed 1.4: a powerful tool for reduction of multiloop integrals, J. Phys. Conf. Ser. 523 (2014) 012059. doi:10.1088/1742-6596/523/1/012059. arXiv:1310.1145.
- [10] R. N. Lee, LiteRed2 MATHEMATICA package, <https://github.com/rnlg/LiteRed2/>, 2019.

- [11] A. V. Smirnov, Algorithm FIRE – Feynman Integral REduction, JHEP 10 (2008) 107. doi:10.1088/1126-6708/2008/10/107. arXiv:0807.3243.
- [12] A. V. Smirnov, V. A. Smirnov, FIRE4, LiteRed and accompanying tools to solve integration by parts relations, Comput. Phys. Commun. 184 (2013) 2820–2827. doi:10.1016/j.cpc.2013.06.016. arXiv:1302.5885.
- [13] A. V. Smirnov, FIRE5: a C++ implementation of Feynman Integral REduction, Comput. Phys. Commun. 189 (2015) 182–191. doi:10.1016/j.cpc.2014.11.024. arXiv:1408.2372.
- [14] A. V. Smirnov, F. S. Chuharev, FIRE6: Feynman Integral REduction with Modular Arithmetic, Comput. Phys. Commun. 247 (2020) 106877. doi:10.1016/j.cpc.2019.106877. arXiv:1901.07808.
- [15] P. Maierhöfer, J. Usovitsch, P. Uwer, Kira—A Feynman integral reduction program, Comput. Phys. Commun. 230 (2018) 99–112. doi:10.1016/j.cpc.2018.04.012. arXiv:1705.05610.
- [16] P. Maierhöfer, J. Usovitsch, Kira 1.2 Release Notes (2018). arXiv:1812.01491.
- [17] J. Klappert, F. Lange, P. Maierhöfer, J. Usovitsch, Integral reduction with Kira 2.0 and finite field methods, Comput. Phys. Commun. 266 (2021) 108024. doi:10.1016/j.cpc.2021.108024. arXiv:2008.06494.
- [18] O. V. Tarasov, Computation of Grobner bases for two loop propagator type integrals, Nucl. Instrum. Meth. A 534 (2004) 293–298. doi:10.1016/j.nima.2004.07.104. arXiv:hep-ph/0403253.
- [19] V. P. Gerdt, D. Robertz, A Maple package for computing Grobner bases for linear recurrence relations, Nucl. Instrum. Meth. A 559 (2006) 215–219. doi:10.1016/j.nima.2005.11.171. arXiv:cs/0509070.
- [20] A. V. Smirnov, V. A. Smirnov, Applying Grobner bases to solve reduction problems for Feynman integrals, JHEP 01 (2006) 001. doi:10.1088/1126-6708/2006/01/001. arXiv:hep-lat/0509187.

- [21] A. V. Smirnov, An Algorithm to construct Grobner bases for solving integration by parts relations, JHEP 04 (2006) 026. doi:10.1088/1126-6708/2006/04/026. arXiv:hep-ph/0602078.
- [22] A. V. Smirnov, V. A. Smirnov, S-bases as a tool to solve reduction problems for Feynman integrals, Nucl. Phys. B Proc. Suppl. 160 (2006) 80–84. doi:10.1016/j.nuclphysbps.2006.09.032. arXiv:hep-ph/0606247.
- [23] R. N. Lee, Group structure of the integration-by-part identities and its application to the reduction of multiloop integrals, JHEP 07 (2008) 031. doi:10.1088/1126-6708/2008/07/031. arXiv:0804.3008.
- [24] M. Barakat, R. Brüser, C. Fieker, T. Huber, J. Piclum, Feynman integral reduction using Gröbner bases, JHEP 05 (2023) 168. doi:10.1007/JHEP05(2023)168. arXiv:2210.05347.
- [25] X. Guan, X. Li, Y.-Q. Ma, Exploring the linear space of Feynman integrals via generating functions, Phys. Rev. D 108 (2023) 034027. doi:10.1103/PhysRevD.108.034027. arXiv:2306.02927.
- [26] C. Hu, B. Feng, J. Shen, Y. Zhang, General one-loop generating function by integration-by-part relations, Phys. Rev. D 111 (2025) 076026. doi:10.1103/PhysRevD.111.076026. arXiv:2403.16040.
- [27] B. Feng, X. Li, Y. Liu, Y.-Q. Ma, Y. Zhang, Symbolic Reduction of Multi-loop Feynman Integrals via Generating Functions (2025). arXiv:2509.21769.
- [28] D. A. Kosower, Direct Solution of Integration-by-Parts Systems, Phys. Rev. D 98 (2018) 025008. doi:10.1103/PhysRevD.98.025008. arXiv:1804.00131.
- [29] S. Smith, M. Zeng, Feynman Integral Reduction using Syzygy-Constrained Symbolic Reduction Rules (2025). arXiv:2507.11140.
- [30] J. Gluza, K. Kajda, D. A. Kosower, Towards a Basis for Planar Two-Loop Integrals, Phys. Rev. D 83 (2011) 045012. doi:10.1103/PhysRevD.83.045012. arXiv:1009.0472.

- [31] R. M. Schabinger, A New Algorithm For The Generation Of Unitarity-Compatible Integration By Parts Relations, JHEP 01 (2012) 077. doi:10.1007/JHEP01(2012)077. arXiv:1111.4220.
- [32] H. Ita, Two-loop Integrand Decomposition into Master Integrals and Surface Terms, Phys. Rev. D 94 (2016) 116015. doi:10.1103/PhysRevD.94.116015. arXiv:1510.05626.
- [33] K. J. Larsen, Y. Zhang, Integration-by-parts reductions from unitarity cuts and algebraic geometry, Phys. Rev. D 93 (2016) 041701. doi:10.1103/PhysRevD.93.041701. arXiv:1511.01071.
- [34] S. Abreu, F. Febres Cordero, H. Ita, M. Jaquier, B. Page, M. Zeng, Two-Loop Four-Gluon Amplitudes from Numerical Unitarity, Phys. Rev. Lett. 119 (2017) 142001. doi:10.1103/PhysRevLett.119.142001. arXiv:1703.05273.
- [35] S. Abreu, F. Febres Cordero, H. Ita, B. Page, M. Zeng, Planar Two-Loop Five-Gluon Amplitudes from Numerical Unitarity, Phys. Rev. D 97 (2018) 116014. doi:10.1103/PhysRevD.97.116014. arXiv:1712.03946.
- [36] J. Böhm, A. Georgoudis, K. J. Larsen, H. Schönemann, Y. Zhang, Complete integration-by-parts reductions of the non-planar hexagon-box via module intersections, JHEP 09 (2018) 024. doi:10.1007/JHEP09(2018)024. arXiv:1805.01873.
- [37] D. Bendle, J. Böhm, W. Decker, A. Georgoudis, F.-J. Pfreundt, M. Rahn, P. Wasser, Y. Zhang, Integration-by-parts reductions of Feynman integrals using Singular and GPI-Space, JHEP 02 (2020) 079. doi:10.1007/JHEP02(2020)079. arXiv:1908.04301.
- [38] Z. Wu, J. Boehm, R. Ma, H. Xu, Y. Zhang, NeatIBP 1.0, A package generating small-size integration-by-parts relations for Feynman integrals (2023). arXiv:2305.08783.
- [39] Z. Wu, J. Böhm, R. Ma, J. Usovitsch, Y. Xu, Y. Zhang, Performing integration-by-parts reductions using NeatIBP 1.1 + Kira (2025). arXiv:2502.20778.
- [40] B. Page, Q. Song, Critical Points and Syzygies for Feynman Integrals (2025). arXiv:2509.17681.

- [41] X. Liu, Y.-Q. Ma, Determining arbitrary Feynman integrals by vacuum integrals, *Phys. Rev. D* 99 (2019) 071501. doi:10.1103/PhysRevD.99.071501. [arXiv:1801.10523](#).
- [42] X. Guan, X. Liu, Y.-Q. Ma, Complete reduction of integrals in two-loop five-light-parton scattering amplitudes, *Chin. Phys. C* 44 (2020) 093106. doi:10.1088/1674-1137/44/9/093106. [arXiv:1912.09294](#).
- [43] X. Guan, X. Liu, Y.-Q. Ma, W.-H. Wu, Blade: A package for block-triangular form improved Feynman integrals decomposition, *Comput. Phys. Commun.* 310 (2025) 109538. doi:10.1016/j.cpc.2025.109538. [arXiv:2405.14621](#).
- [44] P. Mastrolia, S. Mizera, Feynman Integrals and Intersection Theory, *JHEP* 02 (2019) 139. doi:10.1007/JHEP02(2019)139. [arXiv:1810.03818](#).
- [45] H. Frellesvig, F. Gasparotto, S. Laporta, M. K. Mandal, P. Mastrolia, L. Mattiazzi, S. Mizera, Decomposition of Feynman Integrals by Multivariate Intersection Numbers, *JHEP* 03 (2021) 027. doi:10.1007/JHEP03(2021)027. [arXiv:2008.04823](#).
- [46] P. Kant, Finding linear dependencies in integration-by-parts equations: A Monte Carlo approach, *Comput. Phys. Commun.* 185 (2014) 1473–1476. doi:10.1016/j.cpc.2014.01.017. [arXiv:1309.7287](#).
- [47] A. von Manteuffel, R. M. Schabinger, A novel approach to integration by parts reduction, *Phys. Lett. B* 744 (2015) 101–104. doi:10.1016/j.physletb.2015.03.029. [arXiv:1406.4513](#).
- [48] T. Peraro, Scattering amplitudes over finite fields and multivariate functional reconstruction, *JHEP* 12 (2016) 030. doi:10.1007/JHEP12(2016)030. [arXiv:1608.01902](#).
- [49] P. S. Wang, A p-adic algorithm for univariate partial fractions, in: *Proceedings of the Fourth ACM Symposium on Symbolic and Algebraic Computation, SYMSAC '81*, Association for Computing Machinery, New York, NY, USA, 1981, p. 212–217. URL: <https://doi.org/10.1145/800206.806398>. doi:10.1145/800206.806398.

- [50] M. Monagan, Maximal quotient rational reconstruction: an almost optimal algorithm for rational reconstruction, in: Proceedings of the 2004 International Symposium on Symbolic and Algebraic Computation, ISSAC '04, Association for Computing Machinery, New York, NY, USA, 2004, p. 243–249. URL: <https://doi.org/10.1145/1005285.1005321>. doi:10.1145/1005285.1005321.
- [51] J. von zur Gathen, J. Gerhard, Modern Computer Algebra, 3 ed., Cambridge University Press, 2013.
- [52] M. Abramowitz, I. A. Stegun (Eds.), Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables, volume 55 of *National Bureau of Standards Applied Mathematics Series*, Dover Publications, New York, 1965.
- [53] R. Zippel, Probabilistic algorithms for sparse polynomials, in: E. W. Ng (Ed.), Symbolic and Algebraic Computation, Springer Berlin Heidelberg, Berlin, Heidelberg, 1979, pp. 216–226.
- [54] M. Ben-Or, P. Tiwari, A deterministic algorithm for sparse multivariate polynomial interpolation, in: Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing, STOC '88, Association for Computing Machinery, New York, NY, USA, 1988, p. 301–309. URL: <https://doi.org/10.1145/62212.62241>. doi:10.1145/62212.62241.
- [55] D. Grigoryev, M. Karpinski, M. Singer, Computational complexity of sparse rational function interpolation, Siam Journal on Computing - SIAMCOMP (1994).
- [56] J. de Kleine, M. Monagan, A. Wittkopf, Algorithms for the non-monic case of the sparse modular gcd algorithm, in: Proceedings of the 2005 International Symposium on Symbolic and Algebraic Computation, ISSAC '05, Association for Computing Machinery, New York, NY, USA, 2005, p. 124–131. URL: <https://doi.org/10.1145/1073884.1073903>. doi:10.1145/1073884.1073903.
- [57] E. Kaltofen, W.-s. Lee, A. A. Lobo, Early termination in ben-or/tiwari sparse interpolation and a hybrid of zippel's algorithm, in: Proceedings of the 2000 International Symposium on Symbolic and Algebraic Computation, ISSAC '00, Association for Computing Machinery, New

- York, NY, USA, 2000, p. 192–201. URL: <https://doi.org/10.1145/345542.345629>. doi:10.1145/345542.345629.
- [58] Q.-L. Huang, X.-S. Gao, Sparse polynomial interpolation with finitely many values for the coefficients, in: V. P. Gerdt, W. Koepf, W. M. Seiler, E. V. Vorozhtsov (Eds.), *Computer Algebra in Scientific Computing*, Springer International Publishing, Cham, 2017, pp. 196–209.
 - [59] A. V. Smirnov, F. S. Chukharev, FIRE6: Feynman Integral REduction with modular arithmetic, *Comput. Phys. Commun.* 247 (2020) 106877. doi:10.1016/j.cpc.2019.106877. arXiv:1901.07808.
 - [60] T. Peraro, FiniteFlow: multivariate functional reconstruction using finite fields and dataflow graphs, *JHEP* 07 (2019) 031. doi:10.1007/JHEP07(2019)031. arXiv:1905.08019.
 - [61] J. Klappert, F. Lange, Reconstructing rational functions with FireFly, *Comput. Phys. Commun.* 247 (2020) 106951. doi:10.1016/j.cpc.2019.106951. arXiv:1904.00009.
 - [62] F. Lange, J. Usovitsch, Z. Wu, Kira 3: integral reduction with efficient seeding and optimized equation selection, 2025. URL: <https://arxiv.org/abs/2505.20197>. arXiv:2505.20197.
 - [63] A. V. Belitsky, A. V. Smirnov, R. V. Yakovlev, Balancing act: Multivariate rational reconstruction for IBP, *Nucl. Phys. B* 993 (2023) 116253. doi:10.1016/j.nuclphysb.2023.116253. arXiv:2303.02511.
 - [64] A. Smirnov, M. Zeng, Feynman integral reduction: balanced reconstruction of sparse rational functions and implementation on supercomputers in a co-design approach (2024). doi:10.26089/NumMet.2024s03. arXiv:2409.19099.
 - [65] A. V. Smirnov, V. A. Smirnov, How to choose master integrals, *Nucl. Phys. B* 960 (2020) 115213. doi:10.1016/j.nuclphysb.2020.115213. arXiv:2002.08042.
 - [66] J. Usovitsch, Factorization of denominators in integration-by-parts reductions (2020). arXiv:2002.08173.

- [67] V. P. Gerdt, Grobner bases in perturbative calculations, Nucl. Phys. B Proc. Suppl. 135 (2004) 232–237. doi:10.1016/j.nuclphysbps.2004.09.011. arXiv:hep-ph/0501053.
- [68] B. Ruijl, Symbolica, <https://symbolica.io/>, 2023. Accessed: 2023-06-23.
- [69] Z. Bern, E. Herrmann, R. Roiban, M. S. Ruf, A. V. Smirnov, V. A. Smirnov, M. Zeng, Conservative binary dynamics at order $O(\alpha^5)$ in electrodynamics (2023). arXiv:2305.08981.
- [70] Z. Bern, E. Herrmann, R. Roiban, M. S. Ruf, A. V. Smirnov, V. A. Smirnov, M. Zeng, Amplitudes, supersymmetric black hole scattering at $\mathcal{O}(G^5)$, and loop integration, JHEP 10 (2024) 023. doi:10.1007/JHEP10(2024)023. arXiv:2406.01554.
- [71] M. Driesse, G. U. Jakobsen, G. Mogull, J. Plefka, B. Sauer, J. Usovitsch, Conservative Black Hole Scattering at Fifth Post-Minkowskian and First Self-Force Order, Phys. Rev. Lett. 132 (2024) 241402. doi:10.1103/PhysRevLett.132.241402. arXiv:2403.07781.
- [72] M. von Hippel, M. Wilhelm, Refining Integration-by-Parts Reduction of Feynman Integrals with Machine Learning (2025). arXiv:2502.05121.
- [73] Z.-Y. Song, T.-Z. Yang, Q.-H. Cao, M.-x. Luo, H. X. Zhu, Explainable AI-assisted Optimization for Feynman Integral Reduction (2025). arXiv:2502.09544.
- [74] M. Zeng, Reinforcement Learning and Metaheuristics for Feynman Integral Reduction (2025). arXiv:2504.16045.
- [75] Z. Bern, E. Herrmann, R. Roiban, M. S. Ruf, A. V. Smirnov, V. A. Smirnov, M. Zeng, Second-order self-force potential-region binary dynamics at $O(G^5)$ in supergravity (2025). arXiv:2509.17412.
- [76] V. Voevodin, A. Antonov, D. Nikitenko, P. Shvets, S. Sobolev, I. Sidorov, K. Stefanov, V. Voevodin, S. Zhumatiy, Supercomputer Lomonosov-2, “large scale, deep monitoring and fine analytics for the user community”, Supercomputing Frontiers and Innovations 6 (2019) 4–11. doi:10.14529/jsfi190201.